

Project Status Update

Scalable Ride Matching System for a Driverless Taxi Fleet

Duong Bao Huy Tran | Student ID: s224217463 | 29 September 2026

1. Overall project status

- **Current stage:** a working single-vehicle cloud prototype. One ride has been demonstrated from request through matching, dispatch, pickup, drop-off and trip storage.
- **Original goal:** a scalable driverless-taxi platform that ingests telemetry securely, assigns one available vehicle per rider, records trips, and maintains performance as the fleet grows from 500 to 5,000 vehicles.
- **Evidence included:** page 3 contains two original MongoDB Atlas captures showing live and historical telemetry. Seven local unit tests passed again on 29 September. The working prototype does not yet establish the five-second matching target under load, automatic scaling, or zero message loss.

2. Current implementation

Vehicle telemetry

- A Node.js simulator publishes vehicle position, speed, battery and status over MQTT/TLS to AWS IoT Core using a device certificate.
- An AWS IoT rule sends telemetry to SQS. A telemetry worker on ECS Fargate validates readings, rejects impossible GPS jumps, suppresses unchanged parked readings, adds an H3 map cell and stores live/history records in MongoDB Atlas.

Ride and trip flow

- A ride request placed on the ride SQS queue is loaded by the ECS matching worker. The worker searches nearby available vehicles, batches requests for two seconds and uses a conditional MongoDB update to claim a vehicle.
- The matcher publishes a command through the AWS IoT data plane. The simulator moves through pickup and drop-off, then publishes trip completion. An IoT rule and trip SQS queue deliver the event to the ECS trip worker, which saves distance, duration and fare in Atlas.

3. Scope of this update

- The table on page 2 compares the implementation with each milestone in the nine-week plan. Page 3 provides database screenshots; pages 4-5 distinguish observed results from unverified targets and list the next evidence needed.

Progress against the project plan

The status below distinguishes implemented code from results demonstrated in the cloud.

Plan milestone	Progress and evidence as at 29 September 2026
Week 1 - requirements and design	Completed in the proposal. Stakeholders, requirements, architecture and data flow were documented. A public GitHub repository has not been verified for this update.
Week 2 - simulators	Implemented at prototype scale. The Node.js simulator models idle, pickup, occupied and idle again. Ride-request and load generators are present. The AWS demonstration used one vehicle; a simulator screenshot remains to be captured.
Week 3 - AWS IoT and MongoDB	Implemented for the prototype. Certificate-based MQTT/TLS and IoT rules were tested. Atlas stores live positions and history (Figures 1-2); a separate MQTT Test Client screenshot remains to be captured.
Week 4 - telemetry processing	Functionality implemented with a change of technology. The Node.js telemetry worker validates readings, rejects impossible jumps, removes repeated parked readings and tags H3 cells. Node-RED is not deployed.
Week 5 - ride matching	Implemented for one worker. MongoDB nearby-vehicle search, two-second batching and conditional vehicle claim are coded. The assignment is currently greedy; zone partitioning remains open.
Week 6 - trip lifecycle	Core flow implemented. Dispatch, simulated pickup and drop-off, and trip records containing distance, duration and fare were demonstrated. ETA, notifications and rebalancing remain open.
Week 7 - cloud deployment	Partially completed. Telemetry, matcher and trip workers run on ECS Fargate with IoT rules and SQS. Deployment was manual in Learner Lab. API deployment, CDK, CI/CD and private networking are not verified.
Week 8 - load and correctness tests	Not completed. Seven local unit tests passed again on 29 September, but the 500-to-5,000 vehicle ramp, concurrent correctness tests and scaling graphs remain.
Week 9 - report and demonstration	In progress. This status report includes two dated database screenshots. The final report, full visual evidence set and demonstration video remain.

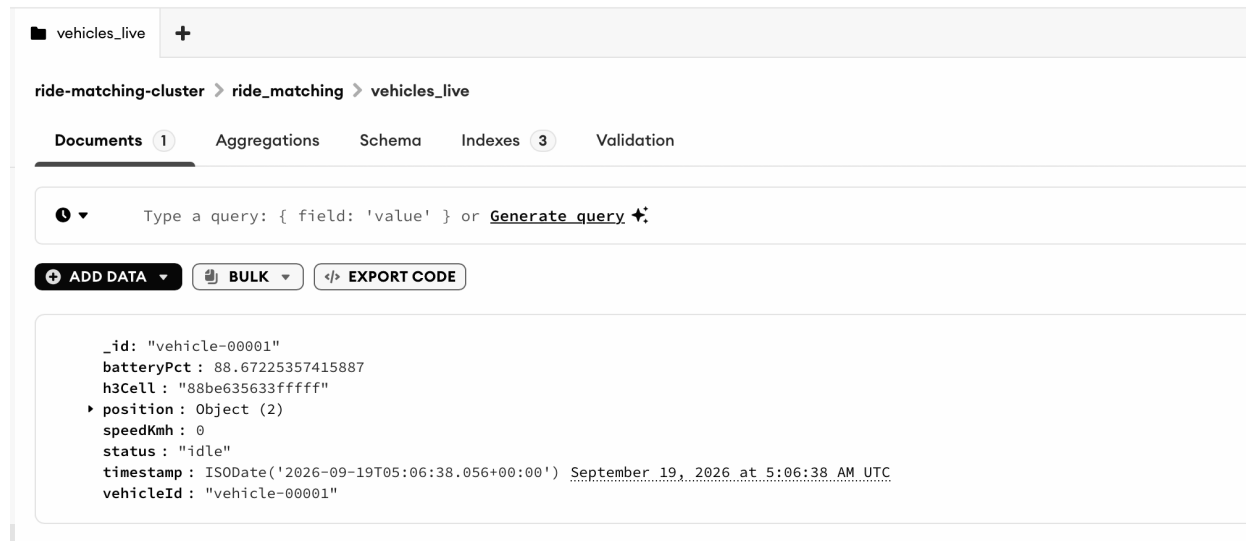
Design changes from the original proposal

- The planned Node-RED preprocessing stage is currently a Node.js telemetry worker on ECS Fargate. The validation, H3 tagging and parked-reading suppression functions exist, but a Node-RED flow does not.
- The verified cloud flow uses IoT rules and direct SQS queues. The proposed SNS fan-out, CDK deployment and city-zone partitioning have not been implemented in the verified prototype.

Visual evidence - database setup

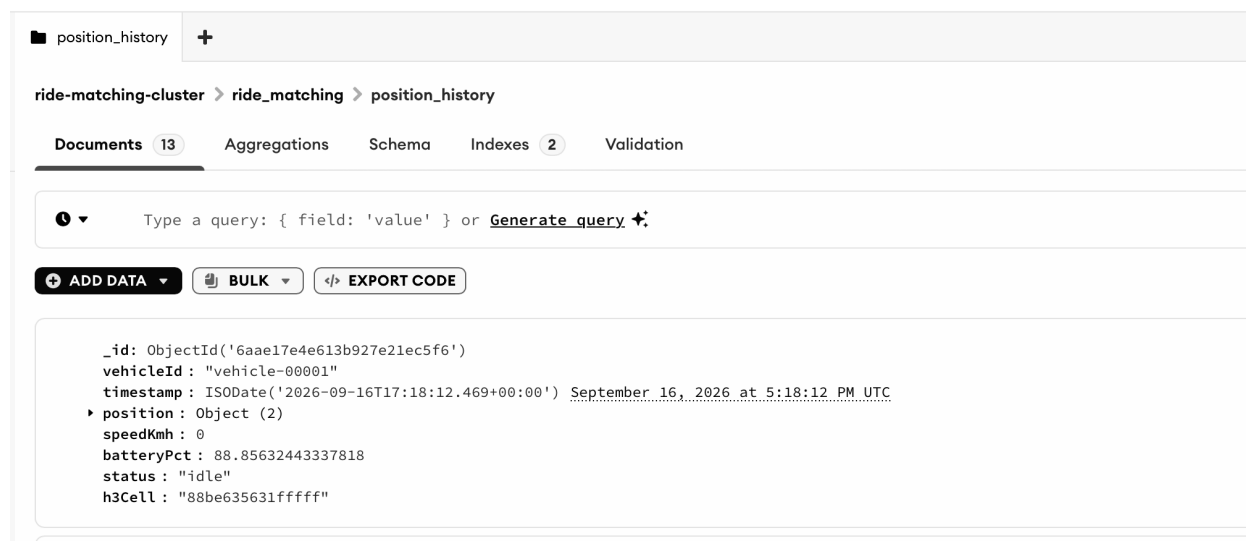
Original MongoDB Atlas screenshots, captured 19 September 2026 and converted to grayscale for this report.

Figure 1. Current vehicle state



Atlas Data Explorer: ride_matching.vehicles_live contains vehicle-00001 with status, speed, battery, timestamp and H3 cell. The capture shows one document.

Figure 2. Historical telemetry



Atlas Data Explorer: ride_matching.position_history shows samples for vehicle-00001 and a total of 13 documents in this capture.

- These captures support the database and telemetry-storage milestones. They do not show throughput, matching latency, or the proposed fleet scale.

Evidence and issues

4. Verification completed

- **Unit tests:** 7 of 7 passed when rerun on 29 September 2026. They cover geographic distance/H3, telemetry bounds and movement checks, repeated parked readings, and one-to-one assignments within a batch.
- **Secure telemetry:** during the AWS walkthrough, the simulator connected to IoT Core over MQTT/TLS and the ECS telemetry worker logged *telemetry service ready*. Figures 1-2 independently show the resulting Atlas records. The MQTT Test Client and simulator screenshots are not available as files for this report.
- **Ride matching:** a manually submitted SQS ride request produced an ECS log showing one request and one match. The simulator received dispatch; vehicle status and position changed during the trip.
- **Trip completion:** a subsequent simulated ride was observed creating a trip record automatically in MongoDB. This tested the completion topic, IoT rule, trip queue and ECS trip worker together; the trip document screenshot still needs to be saved for the evidence pack.
- **Queues:** telemetry, ride and trip queues have dead-letter queues. The telemetry consumer deletes a message after successful handling. Message retention under a real load spike has not been measured.

5. Issues encountered and resolved

- **ECS database connection:** the first MongoDB secret value was not a valid connection string, and Atlas network access prevented connection. Correcting the secret and access settings allowed the telemetry task to reach ready state.
- **Trip event publishing:** the device policy initially allowed telemetry but not the trip-completed topic. The simulator logged repeated *tick failed* messages. Updating the policy and IoT rule restored automatic completion.
- **Stale matcher retries:** a request stayed in matcher memory after its database status changed, causing repeated *requests: 1, matched: 0* logs. The worker now reloads requested rides from MongoDB for each batch. A later run showed one request and one match.

6. What the tests do not yet prove

- A one-vehicle demonstration and the two database captures support the functional path at small scale. They do not prove the high-load latency target, reliable recovery while ECS scales, or safe operation across many concurrent matching workers. Node-RED is not deployed and therefore no Node-RED setup screenshot can be shown.

Remaining work and next steps

7. Open gaps and risks

- **Scale:** only one vehicle and a small number of rides have been verified on AWS. The planned 500-to-5,000 ramp, queue-drain results, latency percentiles and CloudWatch task-count graphs have not been recorded.
- **Matching correctness:** the conditional MongoDB claim helps prevent two simultaneous assignments to one vehicle. The claim-and-dispatch sequence is not transactional; duplicate delivery, service restart and competing workers need explicit tests.
- **Architecture:** Node-RED, SNS fan-out, zone partitioning, ETA calculation, notifications, demand-based rebalancing, S3 history archive, time-series storage, CDK and CI/CD remain outside the verified cloud prototype.
- **Security and operations:** Learner Lab restrictions required the existing LabRole, and Atlas was made reachable from the lab environment. Private networking, dedicated least-privilege roles and scale-down cost behaviour are not yet demonstrated.
- **REST API:** ride creation and fleet/trip query endpoints exist in the repository, but an ECS API deployment and an HTTP-based AWS ride test have not been confirmed.

8. Planned next steps

- **Priority 1 - visual evidence:** save dated screenshots of the simulator publishing, MQTT Test Client receiving, IoT rule/SQS messages, ECS service logs and a completed trip record. Label what each screenshot proves.
- **Priority 2 - functional hardening:** deploy and verify the REST API, then test duplicate messages, a restart after vehicle claim, and two ride requests competing for one vehicle.
- **Priority 3 - scale:** configure autoscaling, rehearse with a smaller fleet, then run the 500-to-5,000 profile if Learner Lab quotas permit. Capture CloudWatch CPU, task count, SQS backlog and matching latency.
- **Priority 4 - proposal alignment:** implement Node-RED or explicitly justify the Node.js replacement, and document other architecture and deployment changes in the final report.

9. Overall assessment

- The main vehicle-to-cloud-to-vehicle ride loop works at small scale. The remaining project work is to measure scalability, test failure cases and make deployment repeatable before claiming that the original non-functional targets have been met.

Sources used for this update

- Project proposal and nine-week plan (26 July 2026); SIT314 4.2D task brief; project source code and 29 September unit-test run; original Atlas screenshots and manual AWS development observations from 19 September.